

Clasificación de enfermedades en hojas de plantas mediante transfer learning: comparación de arquitecturas livianas y explicabilidad con Grad-CAM

BETTIG, Santiago Federico
CEIA - FIUBA
Universidad de Buenos Aires
Buenos Aires, Argentina
santiagobettig@gmail.com

RUBIOLO, Pedro Ignacio
CEIA - FIUBA
Universidad de Buenos Aires
Buenos Aires, Argentina
rubiolopedroignacio@gmail.com

DIEGUEZ, Manuel
CEIA - FIUBA
Universidad de Buenos Aires
Buenos Aires, Argentina
manueldieguetz@gmail.com

TOLOTTO, Lourdes
CEIA - FIUBA
Universidad de Buenos Aires
Buenos Aires, Argentina
tolottolourdes2309@gmail.com

Resumen—[PENDIENTE: Resumen (150–250 palabras). Se redacta al final, cuando estén todos los resultados.]

Palabras clave—clasificación de imágenes, enfermedades de plantas, transfer learning, PlantVillage, EfficientNet, MobileNetV3, Grad-CAM

I. INTRODUCCIÓN

Las plagas y enfermedades de los cultivos son una de las principales amenazas para la seguridad alimentaria. Una evaluación global sobre cinco cultivos básicos estimó que las pérdidas de rendimiento atribuibles a patógenos y plagas alcanzan, en el caso de la papa, un 17,2 % en promedio (8,1–21,0 % según la región), y que las pérdidas más altas se concentran en regiones con déficit de alimentos [1]. El manejo de estas enfermedades depende de un diagnóstico temprano, que todavía se apoya mayormente en la inspección visual realizada por especialistas: un recurso escaso y costoso, sobre todo para productores pequeños y alejados de los centros técnicos.

El aprendizaje profundo y la masificación de los teléfonos inteligentes abrieron la posibilidad de automatizar ese diagnóstico a partir de una fotografía de la hoja. El dataset PlantVillage [2] fue el punto de partida de esta línea: sobre 54 306 imágenes de 14 especies, una red convolucional alcanzó un 99,35 % de accuracy en un conjunto de test separado [3]. Sin embargo, el mismo trabajo reportó que el accuracy caía al 31,4 % sobre imágenes obtenidas de otras fuentes. Estudios posteriores mostraron que PlantVillage contiene sesgos de captura que un modelo puede explotar (por ejemplo, un clasificador entrenado con solo 8 píxeles del fondo alcanza un 49 % de accuracy [4]) y propusieron conjuntos de imágenes tomadas

en condiciones reales, como PlantDoc [5]. En consecuencia, un accuracy cercano al 100 % en PlantVillage no garantiza por sí solo un modelo útil en el campo, y su evaluación debe complementarse con controles de fuga de datos, explicabilidad y pruebas fuera de la distribución de entrenamiento.

Un caso real que ilustra tanto el potencial como esta brecha es PlantVillage Nuru, una aplicación móvil que ejecuta sin conexión un modelo de detección de síntomas foliares en mandioca [6]. En una evaluación de campo en Tanzania, Nuru alcanzó un 65 % de accuracy en el diagnóstico de enfermedades virales, por encima de los extensionistas agrícolas (40–58 %) y de los productores (18–31 %), y llegó al 74–88 % cuando el diagnóstico combinaba seis hojas por planta [7]. Estos resultados muestran que un modelo liviano ejecutado en el teléfono puede aportar valor real, pero también que el desempeño en el campo es muy inferior al que se obtiene en el laboratorio.

En este trabajo desarrollamos un prototipo de clasificación de enfermedades foliares en tres cultivos de importancia hortícola (pimiento, papa y tomate), con 15 clases que incluyen hojas sanas y enfermas de PlantVillage. Partimos de modelos preentrenados en ImageNet y priorizamos arquitecturas livianas (MobileNetV3 [8] y EfficientNet-B0 [9]), compatibles con una inferencia en CPU o en dispositivos móviles. Las contribuciones principales son:

- un protocolo de partición estratificada y agrupada por imágenes casi duplicadas, que reduce la fuga de datos entre entrenamiento y test;
- una comparación sistemática entre *linear probe* y *fine-tuning* parcial de distintos backbones, con y sin data augmentation;

- un análisis cuantitativo de explicabilidad con Grad-CAM [10], que mide qué fracción de la atención del modelo recae sobre la hoja y no sobre el fondo;
- una evaluación de robustez ante perturbaciones de la imagen y sobre datos externos a PlantVillage, junto con la medición del costo de inferencia en CPU.

El resto del trabajo se organiza de la siguiente manera: la Sección II describe los datos, la partición, las arquitecturas y el protocolo experimental; la Sección III presenta y analiza los resultados; y la Sección IV resume las conclusiones y las líneas de trabajo futuro.

II. METODOLOGÍA, DISEÑO Y DESARROLLO

II-A. Dataset

Utilizamos el subconjunto de PlantVillage [2] correspondiente a pimiento, papa y tomate: 20 638 imágenes RGB de 256×256 píxeles distribuidas en 15 clases (12 enfermedades y 3 clases de hoja sana). Las imágenes fueron tomadas en condiciones de laboratorio, con una única hoja centrada sobre un fondo uniforme. El dataset está desbalanceado (Fig. 1a): la clase mayoritaria (virus del rizado amarillo del tomate, TYLCV) tiene 3 208 imágenes y la minoritaria (papa sana) 152, una relación de 21:1. Las hojas sanas representan el 15,6 % del total. Por este desbalance adoptamos el macro-F1 como métrica principal.

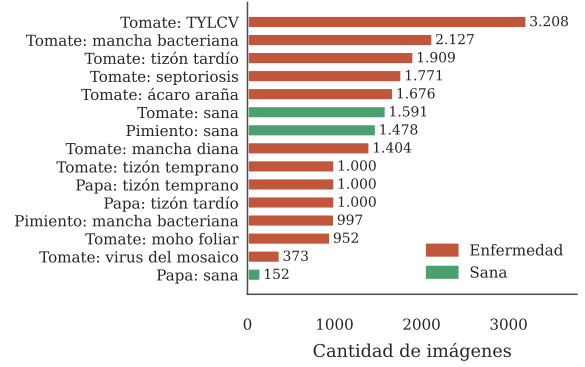
II-B. Partición de datos y control de fuga

PlantVillage incluye varias fotografías de una misma hoja, por lo que una partición aleatoria puede ubicar imágenes casi idénticas en entrenamiento y en test e inflar las métricas. Para detectarlas calculamos un hash criptográfico (MD5) de cada archivo y un hash perceptual (dHash de 64 bits). Encontramos 28 imágenes en 14 grupos de duplicados, todos con etiquetas consistentes. Luego realizamos una partición estratificada por clase y *agrupada* por dHash (StratifiedGroupKFold, semilla fija), de modo que las imágenes de un mismo grupo caen siempre en el mismo subconjunto. El resultado es una partición 71,4/14,3/14,3 % (14 740 imágenes de entrenamiento, 2 949 de validación y 2 949 de test), sin grupos compartidos entre subconjuntos. Esta partición se versiona en el repositorio y es la misma para todos los experimentos. Como el dHash no es invariante a rotaciones, este control no elimina la fuga entre fotografías rotadas de una misma hoja; lo consideramos una limitación del estudio.

II-C. Preprocesamiento y data augmentation

Las imágenes se normalizan con la media y el desvío de ImageNet, que son los usados para preentrenar los backbones. Validación y test usan siempre un preprocesamiento determinístico (solo redimensionado). El data augmentation se aplica únicamente en entrenamiento y combina recortes aleatorios (escala 0,6 - 1,0), volteos horizontales y verticales, rotaciones de hasta $\pm 30^\circ$ ($p = 0,5$), variaciones moderadas de color (brillo y contraste ± 30 %, saturación ± 20 %, tono $\pm 0,03$) y desenfoque gaussiano ($p = 0,2$) (Fig. 1b). Las hojas no tienen una orientación canónica, por lo que las transformaciones

(a) Imágenes por clase



(b) Hoja original (recuadro azul) y siete muestras aumentadas



Figura 1. (a) Cantidad de imágenes por clase en el subconjunto de PlantVillage utilizado. (b) Una hoja de entrenamiento (tizón temprano del tomate) y siete muestras generadas por el pipeline de data augmentation usado en el fine-tuning (160×160).

geométricas son válidas. En cambio, las variaciones de color se limitaron porque el color es en sí mismo una señal diagnóstica (clorosis, necrosis).

II-D. Arquitecturas y estrategia de transferencia

Elegimos backbones preentrenados en ImageNet con distintos compromisos entre costo y capacidad: MobileNetV3-Small y MobileNetV3-Large [8], EfficientNet-B0 [9] y, como referencia sin optimización para dispositivos móviles, ResNet18. Como el entrenamiento se realizó íntegramente en CPU, seguimos una estrategia en dos etapas:

1. **Linear probe:** con el backbone congelado, extraemos una única vez los embeddings de todas las imágenes a 224×224 y entrenamos sobre ellos una capa lineal. Esta etapa, de pocos segundos por modelo, mide la calidad de las representaciones de ImageNet para este dominio y permite comparar las cuatro arquitecturas a bajo costo. También evaluamos una entropía cruzada ponderada por la frecuencia inversa de cada clase.
2. **Fine-tuning parcial:** para las dos arquitecturas con mayor macro-F1 en validación en la etapa anterior (EfficientNet-B0 y MobileNetV3-Small, esta última además la de menor costo), reemplazamos el clasificador y reentrenamos las últimas etapas del backbone (6 bloques en MobileNetV3-Small y 3 en EfficientNet-B0,

Tabla I
CONFIGURACIÓN DE ENTRENAMIENTO

	Linear probe	Fine-tuning
Resolución de entrada	224 × 224	160 × 160
Capas entrenadas	capa lineal	últimas etapas + cabeza
Optimizador	AdamW	AdamW
Learning rate ^a	3×10^{-3}	$10^{-3} / 3 \times 10^{-4}$
Weight decay	10^{-3}	10^{-4}
Scheduler	coseno	coseno
Batch / épocas	256 / 60	32 / 8
Pérdida	CE y CE ponderada	CE
Selección del modelo	máximo macro-F1 en validación	

^aEn fine-tuning: cabeza / backbone.

con 1,39M y 3,17M parámetros entrenables, respectivamente). Las BatchNorm de las etapas congeladas se mantienen en modo evaluación para no alterar sus estadísticas. Cada arquitectura se entrena con y sin data augmentation, en un diseño 2×2 en el que todo lo demás (semilla, partición, resolución, optimizador y número de épocas) permanece fijo.

II-E. Protocolo de entrenamiento y métricas

La Tabla I resume los hiperparámetros de ambas etapas. Para reducir el costo en CPU, el fine-tuning se realiza a 160×160 píxeles. En las dos etapas, la mejor época se elige por macro-F1 en validación, y el conjunto de test se evalúa una única vez con el modelo seleccionado. Reportamos accuracy, balanced accuracy y macro-F1, además del F1 por clase y la matriz de confusión para el análisis de errores. Como métricas de costo registramos el tiempo de entrenamiento, el número de parámetros y la latencia de inferencia en CPU.

II-F. Explicabilidad con Grad-CAM

Para auditar en qué regiones de la imagen se basa el mejor modelo aplicamos Grad-CAM [10] sobre la última capa convolucional del backbone (un mapa de 5×5 a la resolución de entrada), calculado para la clase predicha. Además del análisis visual, cuantificamos la atención del modelo sobre todo el conjunto de test. Para cada imagen segmentamos la hoja aprovechando el fondo uniforme: estimamos el color del fondo en el borde de la imagen y aplicamos un umbral de Otsu sobre la distancia cromática a ese color en el plano (a^*, b^*) de CIELAB, seguido de limpieza morfológica y relleno de huecos. Usar solo la cromaticidad evita que las sombras se confundan con la hoja y conserva las lesiones oscuras. Con la máscara M resultante y el mapa Grad-CAM normalizado L calculamos la fracción de la activación que recae sobre la hoja:

$$E_{\text{hoja}} = \frac{\sum_{i,j} L_{ij} M_{ij}}{\sum_{i,j} L_{ij}}, \quad (1)$$

Como un mapa uniforme obtendría un valor igual al área relativa de la hoja, también reportamos el cociente entre E_{hoja} y esa área (mayor que 1 si la atención se concentra en la hoja más de lo esperable por azar). Por último, aplicamos el *pointing game*, que mide el porcentaje de imágenes cuyo

Tabla II
LINEAR PROBE SOBRE BACKBONES CONGELADOS (ENTROPÍA CRUZADA SIN PONDERAR)

Backbone	Parám. (M)	Extracción (img/s)	Macro-F1		Bal. acc. test
			val	test	
MobileNetV3-Small	0,9	208,6	0,972	0,960	0,962
MobileNetV3-Large	3,0	91,1	0,967	0,960	0,957
EfficientNet-B0	4,0	62,7	0,974	0,959	0,961
ResNet18	11,2	55,4	0,943	0,942	0,941

máximo de activación cae dentro de la hoja. Estas métricas permiten detectar si el modelo explota el fondo, un sesgo conocido de PlantVillage [4], en lugar de los síntomas.

II-G. Evaluación de robustez y con datos externos

[PENDIENTE: Perturbaciones aplicadas al test (tipos y niveles), dataset externo utilizado, mapeo de clases y protocolo de evaluación.]

III. RESULTADOS EXPERIMENTALES Y ANÁLISIS

III-A. Comparación de backbones (linear probe)

La Tabla II resume el desempeño de una capa lineal entrenada sobre los embeddings congelados de cada backbone. Sin modificar ningún peso preentrenado, todas las arquitecturas superan el 94 % de macro-F1 en test, lo que confirma que las representaciones de ImageNet transfieren bien a este dominio. Las arquitecturas diseñadas para dispositivos móviles superan a ResNet18 (macro-F1 0,960 contra 0,942) a pesar de tener entre 4 y 12 veces menos parámetros. Entre ellas, las diferencias en test son menores a 0,002, del orden de la variabilidad entre validación y test: EfficientNet-B0 obtiene el mayor macro-F1 en validación y MobileNetV3-Small, uno prácticamente igual con una extracción 3,3 veces más rápida. La entropía cruzada ponderada no mostró un beneficio consistente: modificó el macro-F1 en test entre $-0,007$ y $+0,004$ según el modelo, por lo que no la utilizamos en el fine-tuning.

III-B. Fine-tuning con y sin data augmentation

El fine-tuning parcial mejora de forma sustancial el linear probe (Tabla III). El macro-F1 en test pasa de 0,960 a 0,986 en MobileNetV3-Small y de 0,959 a 0,994 en EfficientNet-B0, lo que equivale a reducir la fracción de error $(1 - \text{macro-F1})$ un 65 % y un 85 %, respectivamente. Adaptar las últimas etapas del backbone resulta, entonces, mucho más determinante que la elección de la arquitectura en la etapa de linear probe. EfficientNet-B0 supera a MobileNetV3-Small en todas las métricas, con la mitad de errores, pero su entrenamiento demora tres veces más.

El data augmentation no mejoró el desempeño en el test de PlantVillage: las diferencias entre las corridas con y sin augmentation son de a lo sumo 0,002 en macro-F1 (6 imágenes de 2949), un margen que no consideramos significativo con una sola semilla. Sí se observa su efecto regularizador: sin augmentation, EfficientNet-B0 termina con una pérdida de entrenamiento de 0,003 frente a 0,016 en validación (memoriza

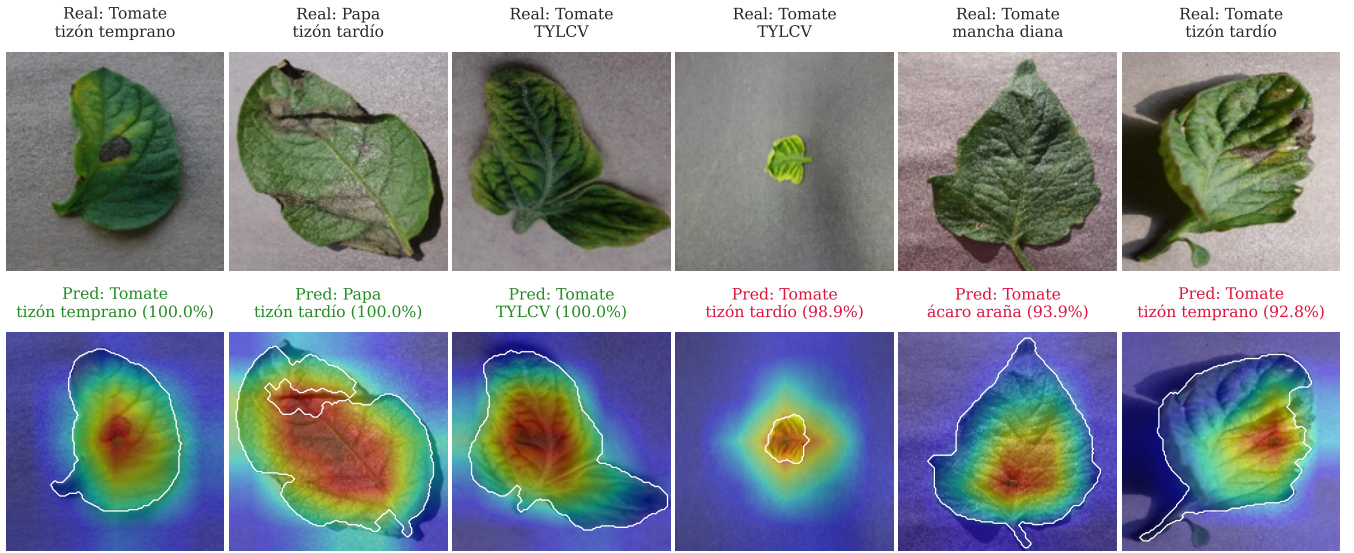


Figura 2. Grad-CAM del modelo EfficientNet-B0 con augmentation sobre imágenes de test. Las tres columnas de la izquierda muestran aciertos típicos (imagen con la mediana de E_{hoja} dentro de su clase) y las tres de la derecha, los errores con mayor confianza. El contorno blanco indica la máscara de la hoja usada para calcular E_{hoja} .

Tabla III
FINE-TUNING PARCIAL A 160×160 (8 ÉPOCAS, CPU)

Modelo	Aug.	Parám. entren. (M)	F1 val	F1 test	Errores test	Tiempo (min)
MobileNetV3-S	no	1,39	0,990	0,986	24	10,7
MobileNetV3-S	sí	1,39	0,987	0,986	30	10,6
EfficientNet-B0	no	3,17	0,995	0,994	11	33,8
EfficientNet-B0	sí	3,17	0,995	0,993	17	32,4

el conjunto de entrenamiento), mientras que con augmentation la pérdida de entrenamiento (0,038) queda por encima de la de validación (0,019). Como el test proviene de la misma distribución que el entrenamiento, es esperable que el beneficio del augmentation aparezca recién ante condiciones de captura distintas (Secciones III-E y III-F). Por último, en las cuatro corridas el macro-F1 de validación todavía crecía en la última época, por lo que un entrenamiento más largo podría mejorar levemente estos valores.

III-C. Análisis por clase y de errores

Con el modelo seleccionado (EfficientNet-B0 con augmentation, el de mayor macro-F1 en validación), todas las clases superan un F1 de 0,975. Las más bajas son tizón temprano del tomate (0,975) y papa sana (0,976). Esta última es la clase con menor F1 en tres de las cuatro corridas, lo que es coherente con que sea la clase minoritaria (21 imágenes de test). De los 17 errores, 16 ocurren entre enfermedades de una misma especie y solo uno confunde una hoja sana con una enferma. Las confusiones más frecuentes se dan entre enfermedades del tomate con síntomas visualmente parecidos: tizón temprano contra tizón tardío, y mancha diana contra ácaro araña o moho foliar. Estos mismos pares ya aparecían como los principales

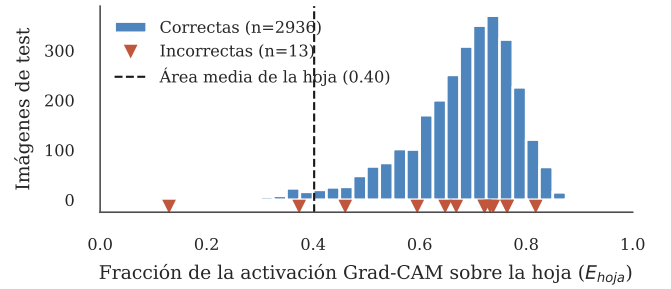


Figura 3. Distribución de E_{hoja} (fracción de la activación Grad-CAM sobre la hoja) en las 2949 imágenes de test. Los triángulos marcan los 13 errores; la línea punteada, el área media que ocupa la hoja.

errores del linear probe, lo que sugiere una ambigüedad propia de los síntomas más que una limitación de un modelo en particular.

III-D. Explicabilidad (Grad-CAM)

El análisis de Grad-CAM se realizó sobre un reentrenamiento del mismo modelo (macro-F1 en test de 0,994; 13 errores). En el conjunto de test completo, el 67,8 % de la activación recae sobre la hoja, que ocupa en promedio el 40,2 % de la imagen: la atención está 1,77 veces más concentrada en la hoja de lo que se esperaría por azar. Además, el máximo de activación cae dentro de la hoja en el 97,0 % de las imágenes (Fig. 3). Los aciertos de la Fig. 2 muestran que el modelo se enfoca en las lesiones necróticas y en las zonas cloróticas.

Los errores no se explican por una atención desviada hacia el fondo: su E_{hoja} medio (0,63) y su *pointing game* (92 %) son apenas menores que los de los aciertos (0,68 y 97 %). Es decir, el modelo mira la hoja pero interpreta mal síntomas

Tabla IV
COSTO DE INFERENCIA EN CPU (160×160 , FP32)

Modelo	Parám. (M)	Tamaño (MB)	Latencia bs=1 media (p95) [ms]	Throughput bs=32 [img/s]
MobileNetV3-Small	1,53	5,9	7,7 (9,1)	920
EfficientNet-B0	4,03	15,5	18,0 (25,8)	152

ambiguos. La excepción es una hoja de TYLCV de tamaño muy reducido, en la que la activación se extiende sobre el fondo (Fig. 2, cuarta columna). Las clases con menor E_{hoja} son el virus del mosaico (0,54) y el tizón tardío del tomate (0,55), cuyas hojas suelen estar enrolladas o deformadas. Parte de la activación fuera de la hoja se debe a la baja resolución del mapa (5×5), que al interpolarse se extiende sobre los bordes. Aun así, que cerca de un tercio de la activación caiga fuera de la hoja indica que el fondo no es completamente irrelevante para el modelo, en línea con el sesgo descrito en [4].

III-E. Robustez ante perturbaciones

III-F. Evaluación con datos externos

III-G. Costo computacional

La Tabla IV compara el costo de inferencia de los dos modelos ajustados en una CPU de notebook (Intel Core i7-1355U), con entrada de 160×160 . Ambos permiten clasificar en tiempo real sin GPU: EfficientNet-B0 procesa una imagen en 18 ms y MobileNetV3-Small en 7,7 ms, con un modelo 2,6 veces más liviano (5,9 MB). Si se prioriza el desempeño, EfficientNet-B0 es la mejor opción para el prototipo. MobileNetV3-Small, que comete el doble de errores pero es 2,3 veces más rápido, resulta una alternativa adecuada para dispositivos más limitados, como un teléfono de gama baja.

IV. CONCLUSIONES

REFERENCIAS

- [1] S. Savary, L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts, and A. Nelson, "The global burden of pathogens and pests on major food crops," *Nature Ecology & Evolution*, vol. 3, no. 3, pp. 430–439, 2019.
- [2] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv:1511.08060, 2015. [Online]. Available: <https://arxiv.org/abs/1511.08060>
- [3] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [4] M. A. Noyan, "Uncovering bias in the PlantVillage dataset," arXiv:2206.04374, 2022. [Online]. Available: <https://arxiv.org/abs/2206.04374>
- [5] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," in *Proc. 7th ACM IKDD CoDS and 25th COMAD*, 2020, pp. 249–253.
- [6] A. Ramcharan, P. McCloskey, K. Baranowski, N. Mbilinyi, L. Mrisho, M. Ndalawha, J. Legg, and D. P. Hughes, "A mobile-based deep learning model for cassava disease diagnosis," *Frontiers in Plant Science*, vol. 10, p. 272, 2019.
- [7] L. M. Mrisho, N. A. Mbilinyi, M. Ndalawha *et al.*, "Accuracy of a smartphone-based object detection model, PlantVillage Nuru, in identifying the foliar symptoms of the viral diseases of cassava—CMD and CBS," *Frontiers in Plant Science*, vol. 11, p. 590889, 2020.

- [8] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1314–1324.
- [9] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 97, 2019, pp. 6105–6114.
- [10] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 618–626.